

$F2 \parallel wC_{\max}$ Structural Properties and Dynamic Programming

Author One^{a,*}, Author Two^b

^aInstitution One, Location, Country

^bInstitution Two, Location, Country

Abstract

Minimizing $wC_{\max}$ on a two-machine flow shop, denoted $F2 \parallel wC_{\max}$, is a classical NP-hard scheduling problem. Existing exact methods rely on enumerative techniques that scale poorly with instance size, while approximation algorithms often impose restrictive assumptions on processing times. We first sequence all jobs globally by Johnson's rule, obtaining an initial schedule π . Scanning π from left to right, we partition it into at most K blocks: whenever a weight not seen before appears, it serves as the final job of the current block, and the next job starts a new block. We establish structural properties of optimal schedules: the permutation property, the positional constraint that block $\mathcal{G}_k$ jobs cannot appear beyond the first k blocks, and the block-end condition requiring the final job of each block to carry that block's weight (automatically satisfied by construction). The intra-block Johnson ordering inherited from π is further formalized within the DP framework. Building on these properties and the interval-based dynamic programming framework (Hall, 1998) developed for $F2|r_j|C_{\max}$, we design a polynomial-time dynamic programming algorithm with $O(|Y|_{\max} \cdot n)$ complexity that appends jobs block by block. We further show that geometric rounding of weights and processing times converts this algorithm into a polynomial-time approximation scheme.

Keywords: Flow shop scheduling; $wC_{\max}$; Dynamic programming; Structural properties

1 Introduction

2 Preliminaries

We consider the problem $F2 \parallel wC_{\max}$. Let $\mathcal{J} = \{J_1, J_2, \dots, J_n\}$ be the set of n jobs to be processed on two machines M_1 and M_2 in series: each job J_j must be processed first on

*Corresponding author. E-mail address: author@email.com

M_1 for a_j time units and then on M_2 for b_j time units, with no preemption allowed. M_1 and M_2 can process at most one job at a time. Each job J_j has a weight $w_j > 0$. A schedule $\pi = (\pi(1), \pi(2), \dots, \pi(n))$ specifies the processing order on both machines. For a schedule π , let $A_j = \sum_{h=1}^j a_{\pi(h)}$ and $B_j = \sum_{h=1}^j b_{\pi(h)}$ be the cumulative M_1 and M_2 completion times after the first j jobs.

The objective is to find a schedule π that minimizes $wC_{\max}$.

For $F2 \parallel wC_{\max}$, a permutation schedule in which the job order on M_1 and M_2 , is optimal (Baker, 1974). Johnson's rule (Johnson, 1954) sequences all n jobs to minimize the makespan,

Lemma 2.1. *For $F2 \parallel wC_{\max}$, there exists an optimal schedule in which all jobs are sequenced according to Johnson's rule.*

Proof. □

3 NP-hard

Theorem 3.1. *$F2 \parallel wC_{\max}$ is NP-hard.*

Proof. We reduce from the Partition problem: given n positive integers $x_1, x_2, \dots, x_r$ with $\sum_{i=1}^r x_i = 2T$, does there exist subset Z_1 and $Z_2 \subseteq \{1, \dots, r\}$ such that: $\sum_{i \in Z_1} x_i = \sum_{i \in Z_2} x_i = T$?

Construct an instance of $F2 \parallel wC_{\max}$, Let the number of jobs $n = r + 1$. Let $a_j = 1$, $b_j = rx_j$, $w_j = T + 1$ for jobs $j = 1, \dots, n - 1$. Put $a_n = rT$, $b_n = 1$, $w_n = 2T + 1$ for job J_n . Does there exist a threshold Y such that:

$$wC_{\max} \leq Y = r(T + 1)(2T + 1)$$

holds?

We partition jobs j_1 to j_{n-1} into two subsets via j_n , the distribution of jobs on the machine is $(\mathcal{J}_{j \in A_1}, J_n, \mathcal{J}_{j \in A_2})$. Since the processing times of A_1 and A_2 are unknown, while their total processing time is $2T$. We may assume that the processing time of A_1 on M_2 is $r(T + \xi)$, A_2 is $r(T - \xi)$, where $-T \leq \xi \leq T$. Therefore, the completion time of J_n is determined by the following formula:

$$C_n^1 = |A_1| + rT + 1 \tag{1}$$

$$C_n^2 = 1 + r(T + \xi) + 1 \tag{2}$$

and $C_n = \max\{C_n^1, C_n^2\}$

If $\xi = 0$, then $\sum_{j \in A_1} b_j = \sum_{j \in A_2} b_j$, which means the partition problem has a feasible solution. In this case, the completion time of job J_n is determined by (1), we have:

$$\begin{aligned}
w_n C_n &= (2T + 1)(|A_1| + rT + 1) \\
&\leq (2T + 1)(rT + r) \\
&= r(T + 1)(2T + 1) \\
&= Y \\
w C_{\max} &= r(T + 1)(C_n + rT) = Y
\end{aligned}$$

so $w_j C_j \leq w C_{\max} = Y$

If the scheduling problem exists a threshold y such that $w C_{\max} \leq y$. When $\xi > 0$, C_n is determined by (2), we have:

$$\begin{aligned}
w_n C_n &= (1 + r(T + \xi) + 1)(2T + 1) \\
&\geq (1 + r(T + 1) + 1)(2T + 1) \\
&> Y
\end{aligned}$$

When $\xi < 0$, C_n is determined by (1), we have:

$$\begin{aligned}
w C_{\max} &= (C_n + \sum_{j \in A_2} b_j)(T + 1) \\
&= (1 + rT + |A_1| + r(T - \xi))(T + 1) \\
&\geq (rT + r + rT)(T + 1) \\
&= r(T + 1)(2T + 1) = Y
\end{aligned}$$

so $w C_{\max} > Y$, there exists a contradiction.

Therefore, the Partition instance admits a solution if and only if the constructed $F2 \parallel w C_{\max}$ instance admits a schedule with $w C_{\max} \leq Y$. $\square$

4 $F2 \parallel w C_{\max}$ for the number of weights is constant

When the number of distinct weights K is bounded by a constant, the structural properties established in Section 2 enable a polynomial-time dynamic programming algorithm. By Lemma 2.1, the initial job sequence follows Johnson's rule; the block partition and all subsequent steps are defined with respect to this Johnson order. We first describe the block partition and group logic underlying our approach, then present the DP formulation, and finally give the complete algorithm and its complexity analysis.

4.1 Algorithm Properties and Dynamic Programming

The jobs are partitioned into blocks by a single left-to-right scan. Starting from the first job, a new block is created whenever a weight value not encountered before appears; that job becomes the final job of the current block, and the next job starts a new block. Since each block introduces at least one previously unseen weight, the procedure yields at most K blocks, denoted $\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_K$. By construction, the final job of each block carries the weight that triggered the cut. By construction, block weights are non-increasing, i.e., $W(\mathcal{B}_1) \geq W(\mathcal{B}_2) \geq \dots \geq W(\mathcal{B}_K)$. Moreover, in any optimal schedule, jobs belonging to $\mathcal{B}_1 \cup \dots \cup \mathcal{B}_k$ must be scheduled before any job from $\mathcal{B}_{k+1} \cup \dots \cup \mathcal{B}_K$.

Jobs of the same weight form a natural group, denoted $\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_K$. Scheduling heavier groups before lighter ones automatically enforces the non-increasing block-weight condition, and scanning groups in descending weight order directly yields the block partition.

Example 4.1. Consider a 6-job instance with Johnson sequence $\pi = (J_1, \dots, J_6)$ and weights $(w_1, \dots, w_6) = (2, 2, 5, 2, 4, 2)$. Scanning π from left to right and cutting whenever a weight not seen before appears yields the following partition:

$$\underbrace{J_1, J_2, J_3}_{\mathcal{B}_1} \quad \underbrace{J_4, J_5}_{\mathcal{B}_2} \quad \underbrace{J_6}_{\mathcal{B}_3}$$

The resulting blocks are $\mathcal{B}_1 = \{J_1, J_2, J_3\}$, $\mathcal{B}_2 = \{J_4, J_5\}$, $\mathcal{B}_3 = \{J_6\}$. And $\mathcal{G}_1 = \{J_1, J_2, J_4, J_6\}$, $\mathcal{G}_2 = \{J_5\}$, $\mathcal{G}_3 = \{J_3\}$.

Lemma 4.1. For any schedule π partitioned into blocks $\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_K$ as constructed by the scanning procedure, if the jobs within each block $\mathcal{B}_k$ are sequenced according to Johnson's rule, then π minimizes the makespan of each block and consequently yields the minimum $wC_{\max}$ among all schedules respecting the same block partition.

Proof. Consider any block $\mathcal{B}_k$. Its internal job sequence determines the block makespan $C_{(k)}$, which feeds into the CON concatenation to compute the overall completion times. By Johnson's rule (Johnson, 1954), sequencing the jobs within $\mathcal{B}_k$ in Johnson order minimizes $C_{(k)}$. Since the objective $wC_{\max}$ is non-decreasing in each block's makespan, minimizing every $C_{(k)}$ yields the minimum $wC_{\max}$ for the given partition. $\square$

4.1.1 Dynamic Programming Formulation

We now develop a dynamic programming algorithm that processes jobs sequentially in Johnson order and assigns each job to one of the K blocks. The state representation follows the interval-based dynamic programming framework of (Hall, 1998) for $F2|r_j|C_{\max}$, and the concatenation logic follows the CON algorithm of (Kovalyov and Werner, 1997). We denote by $\mathcal{H}$ the set of states maintained during the computation.

State Definition. A state encodes the aggregate parameters of all K blocks after a prefix of jobs has been processed. It is represented as a $4K$ -tuple

$$\mathcal{H} = (C_1, \dots, C_K, A_1, \dots, A_K, B_1, \dots, B_K, L_1, \dots, L_K),$$

where for each block $i = 1, \dots, K$:

- A_i — total processing time on machine M_1 of jobs assigned to block i ;
- B_i — total processing time on machine M_2 of jobs assigned to block i ;
- C_i — makespan of block i when the jobs assigned to it are scheduled in isolation from time zero, computed via the recurrence of equation (??);
- L_i — weight of the last job in block i (or 0 if block i is empty).

The initial state set contains a single state with all components set to zero: $\mathcal{H} = \{(0, \dots, 0)\}$.

State Transition. Suppose that the jobs are indexed in Johnson order as $J_1, J_2, \dots, J_n$. Let $k \in \{1, \dots, K\}$ be the block to which job J_j belongs, as determined by the block partition. Given a state $(C_1, \dots, C_K, A_1, \dots, A_K, B_1, \dots, B_K, L_1, \dots, L_K) \in \mathcal{H}$, adding job J_j to block k updates the block's parameters as follows (see equation (2) in Kovalyov and Werner, 1997):

$$A'_k = A_k + a_j, \tag{3}$$

$$B'_k = B_k + b_j, \tag{4}$$

$$C'_k = \max\{C_k + b_j, A_k + a_j + b_j\}. \tag{5}$$

All other components remain unchanged. If J_j is the last job of block k (i.e., the job whose weight triggered the block cut), then $L'_k \leftarrow w_j$; otherwise $L'_k \leftarrow L_k$. The updated state is added to the new state set $\mathcal{H}'$.

After processing each job, the state set may contain multiple states representing different assignments of jobs to blocks. To keep the state set small, we apply the following dominance rule, which is analogous to the tuple-merging step in Algorithm H of Kovalyov and Werner, 1997.

Lemma 4.2. *Let S and S' be two states in $\mathcal{H}$ after processing the same prefix of jobs. If $A_i \leq A'_i$, $B_i \leq B'_i$, $C_i \leq C'_i$, and $L_i \leq L'_i$ for all $i = 1, \dots, K$, then S' is dominated by S and can be removed from further consideration.*

Proof. Let the remaining jobs be processed identically from both states. By the monotonicity of the recurrence in (??) and the CON concatenation formula, smaller A_i , B_i , C_i values

lead to no larger final makespan. Smaller L_i ensures the weighted completion time objective is not increased. Hence S yields a schedule no worse than S' under $wC_{\max}$. $\square$

CON

- 1: **Input:** Current completion times (T_A, T_B) ; block parameters $(A_{(i)}, B_{(i)}, C_{(i)})$ for block $\mathcal{B}_i$.
 - 2: **Output:** Updated completion times after concatenating $\mathcal{B}_i$.
 - 3: $T_A^{\text{old}} \leftarrow T_A, \quad T_B^{\text{old}} \leftarrow T_B$
 - 4: $T_A \leftarrow T_A^{\text{old}} + A_{(i)}$
 - 5: $T_B \leftarrow \max\{T_A^{\text{old}} + C_{(i)}, T_B^{\text{old}} + B_{(i)}\}$
 - 6: **return** (T_A, T_B)
-

Dynamic Programming

- 1: **Input:** Jobs $\mathcal{J}$ with (a_j, b_j, w_j) , numbered $1, \dots, n$ in Johnson order.
- 2: **Output:** The optimal objective value $wC_{\max}$.
- 3: **Step 1 (Initialization):**
- 4: $\mathcal{H} \leftarrow \{(0, \dots, 0)\}$
- 5: **Step 2 (Generation):**
- 6: **for** $j = 1$ to n **do**
- 7: $k \leftarrow \text{blk}(j)$
- 8: $\mathcal{H}_{\text{new}} \leftarrow \emptyset$
- 9: **for each** $(C_1, \dots, C_K, A_1, \dots, A_K, B_1, \dots, B_K, L_1, \dots, L_K) \in \mathcal{H}$ **do**
- 10: $A'_k \leftarrow A_k + a_j$
- 11: $B'_k \leftarrow B_k + b_j$
- 12: $C'_k \leftarrow \max\{C_k + b_j, A_k + a_j + b_j\}$
- 13: **if** j is the last job of $\mathcal{B}_k$ **then**
- 14: $L'_k \leftarrow w_j$
- 15: **else**
- 16: $L'_k \leftarrow L_k$
- 17: **end if**
- 18: Add $(\dots, C'_k, \dots, \dots, A'_k, \dots, \dots, B'_k, \dots, \dots, L'_k, \dots)$ to $\mathcal{H}_{\text{new}}$
- 19: **end for**
- 20: $\mathcal{H} \leftarrow \text{MERGEANDPRUNE}(\mathcal{H}_{\text{new}})$
- 21: **end for**
- 22: **Step 3 (Evaluation):**
- 23: $wC_{\max} \leftarrow +\infty$

```

24: for each state  $S \in \mathcal{H}$  do
25:    $T_A \leftarrow 0, T_B \leftarrow 0, \text{obj} \leftarrow 0$ 
26:   for  $i = 1$  to  $K$  do
27:      $(T_A, T_B) \leftarrow \text{CON}((T_A, T_B), (A_i, B_i, C_i))$ 
28:      $\text{obj} \leftarrow \max\{\text{obj}, L_i \cdot T_B\}$ 
29:   end for
30:    $wC_{\max} \leftarrow \min\{wC_{\max}, \text{obj}\}$ 
31: end for
32: return  $wC_{\max}$ 

```

4.1.2 Complexity Analysis

Theorem 4.1. *Algorithm 1 computes an schedule for $F2 \parallel wC_{\max}$ in $O()$ time.*

Proof.

□

4.2 FPTAS

5 Conclusion

References

- Baker, K. R. (1974). *Introduction to Sequencing and Scheduling*. Wiley, New York.
- Hall, L. A. (1998). Approximability of flow shop scheduling. *Mathematical Programming*, 82(1-2):175–190.
- Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.
- Kovalyov, M. Y. and Werner, F. (1997). A polynomial approximation scheme for problem $F2/r_j/c_{\max}$. *Operations Research Letters*, 20(2):75–79.